

We understand that phishing detection itself is already well studied, so we hope to identify a meaningful limitation in existing approaches rather than simply compare classifiers. We have identified datasets such as **PhiUSIL** and **PhreshPhish** that seem promising, and some potential directions we are currently exploring include:

- whether **lightweight URL-based features** can retain strong detection performance without requiring richer webpage-content features;
- the **trade-off between predictive performance and the amount/cost of information** required for detection;
- **cross-dataset or temporal robustness**, such as whether a detector continues to perform well on independently collected or newer phishing data; and
- potentially combining these ideas by studying whether a lightweight detector can achieve strong performance **while generalizing well to independent data**.

Identifying Suspicious URLs: An Application of Large-Scale Online Learning

Article from 2009 that creates a lightweight model using URL information to predict phishing links with 99% accuracy. Only lexical host based features. No context. Look at tokens in the position of the sentence they're in (.com out of place or ebay out of place, etc.). They look at IP and geographical information in URLs. Use Perceptron and other linear learning based algs (predict, get result, update next prediction. They use host based features about DNS and stuff. Womp womp.

This paper has good detail in regards to their algs. I think we could replicate quite easily.

Evaluating the Impact of Feature Engineering in Phishing URL Detection: A Comparative Study of URL, HTML, and Derived Features

TABLE 2. The initial parameter for each model.

Model	Parameter
Random Forest	n_estimators=10
Naive Bayes	default
k-Nearest Neighbors	n_neighbors=1
Logistic Regression	default
Decision Tree	max_depth=30
Support Vector Machine	kernel='linear', C=1.0, random_state=42
Gradient Boosting	max_depth=4, learning_rate=0.7
LightGBM	random_state=42
XGBClassifier	max_depth=4, learning_rate=0.7
CatBoost	learning_rate = 0.1

1) URL FEATURES URL features play an essential role in phishing URL detection [32] by offering key insights into a web page's legitimacy. The main focus URL features to be used in this study are IsHTTPS, IsDomainIP, TLD, URLLength, NoOfSubDomain, NoOfDots, NoOfObfuscatedChar, NoOfEqual, NoOfQmark, NoOfAmp, and NoOfDigits.

TABLE 3. ML model performance on URL-based feature.

Model	Accuracy	Precision	Recall	F1 Score
Random Forest	0.9658	0.9704	0.9595	0.9649
Naive Bayes	0.7274	0.9942	0.4471	0.6168
k-Nearest Neighbors	0.9459	0.9418	0.9483	0.9450
Logistic Regression	0.9190	0.9441	0.8874	0.9149
Decision Tree	0.9659	0.9711	0.9591	0.9650
Support Vector Machine	0.9219	0.9583	0.8790	0.9170
Gradient Boosting	0.9675	0.9726	0.9609	0.9667
LightGBM	0.9674	0.9726	0.9606	0.9666
XGBoost	0.9667	0.9728	0.9589	0.9658
CatBoost	0.9673	0.9713	0.9618	0.9665

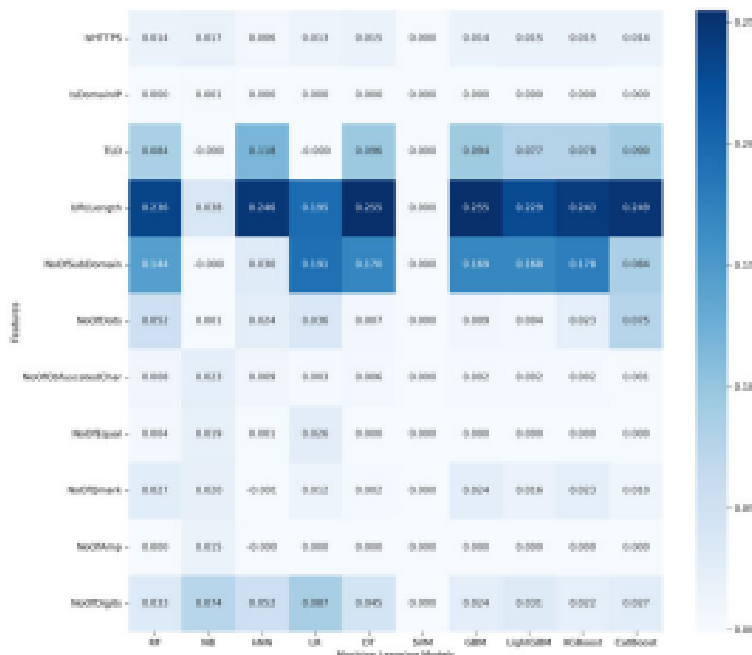


FIGURE 3. Feature importance heatmap of URL-based features across machine learning models.

Idea: super light weight identification only based off of minimal number of parameters? Like length, TLD, and # sub domains? Since they're the most valuable

- **Explainability layer:** build a SHAP/LIME-based explanation on top of your best classifier and evaluate whether the top features it surfaces are stable across the dataset or noisy/dataset-specific — ties into gap #6 and is a good "AI trust" angle if that resonates with your data mining course focus.

Maybe we could create a model for this dataset, and then try to source some like AI based phishing links and test our model against those results and try to identify why AI phishing is thriving?

https://albahnsen.wordpress.com/wp-content/uploads/2018/05/deepphish-simulating-malicious-ai_submitted.pdf

stopwatch.ai could be interesting. Seems like you're allowed to try to generate attacks for different OS' and targets and things

Public phishing corpora with AI-generated components already exist. Groups like APWG, PhishTank, and various academic security labs have published adversarial phishing datasets (including LLM-generated lures) specifically for detector benchmarking. Using an existing, responsibly-curated dataset sidesteps the generation problem entirely.

Red-team via a security research partnership. If this is genuinely for defensive model evaluation, some organizations get access to adversarial datasets through academic collaborations or industry security consortia (e.g., with explicit ethical review and containment), which is a more appropriate channel than an open-ended generation request.

See if we can have this through UVic?

<https://www.pcworld.com/article/2967414/cyber-gangsters-with-ai-how-phishing-emails-become-uncannily-real.html>

Pop culture article just talking about the situation and deepphish

https://albahnsen.wordpress.com/wp-content/uploads/2018/05/deepphish-simulating-malicious-ai_submitted.pdf

deepphish article that the news article was talking about. They look at an existing dataset of phishing links and phishing detection model, and then create a new model that generates phishing links using the most effective "threat actors" using a long-short memory network.

"Using the algorithm with

the data of two threat actors, they were able to improve their effectiveness rate, measured as the percentage of attacks not blocked by a proactive phishing detection system, from 0.69% to 20.9%, and from 4.91% to 36.28%, respectively".

This paper makes me wonder if institutions give feedback to phishing urls so you could train a model on Gmail responses to attempted phishing emails for example...

<https://ieeexplore.ieee.org/abstract/document/10288940>

this link isn't working on my work PC but I think we can access it through UVic? Interesting title "Exploring the Dark Side of AI: Advanced Phishing Attack Design and Deployment Using ChatGPT"

<https://link.springer.com/article/10.1007/s10462-024-10973-2>

Great paper talking about the overall threat and emergence of AI phishing. The whole paper is great background, but paragraph 4.3 specifically pertains to our stuff.

This paper raises a good point that the IT/security industry within businesses are focused more on better human training/awareness than technical identification of phishing content. Could frame our research as "we've built a model to identify phishing links, but can easily generate new URLs using AI to beat our model, and with added reasons of industry data x (showing phishing stuff is on the rise/everlasting) and economic implications a, b, and c, it's rational for organizations to focus security efforts on education over identification due to z% better ROI on education capex vs identification capex. " It isn't a novel contribution, but it's just what I'm thinking through right now. I'll have some finalized/polished ideas ready for our meeting tomorrow.

<https://claude.ai/share/707abc61-6c7d-43f5-a0ed-55856622c2bc>

2. Shift the dependent variable from "does it evade" to "does it fool a human"

Run a small human-subjects arm: take AI-generated URLs that evade your model and show them (alongside real phishing and real benign URLs) to a small sample of people, measuring detection rate. The real finding becomes: are model-evading URLs *also* harder for humans, or is there a divergence — URLs that evade the ML detector but a human would flag instantly (or vice versa)? That divergence is the novel contribution, and it directly informs the capex-vs-training argument with actual data instead of an assumed conclusion.

Could be an interesting conclusion that the model trained urls might beat the model we trained but still lose to a human well? Like they pick up odd artifacts to beat the model?

4. Cost-model the arms race explicitly

Since your original angle was econ-flavored (capex ROI), actually quantify it: estimate the compute/token cost per generated evading URL vs. the cost of retraining the detector vs. estimated cost of a training program per employee per successful catch. Turn "training has better ROI" from an assumed conclusion into a number you derive from your own experiment's cost data. That's the unique spin — most phishing-detection papers don't attach \$ to either side.

Combination of two and 4 could be sick.

Yes, that's the strongest version of this — and it's a genuinely useful finding rather than a foregone conclusion. The mechanism you're pointing at is basically: your detector is a proxy for "phishing-ness," and once you optimize URLs against the proxy instead of the real target (human judgment), you're going to drift off-distribution in ways the proxy doesn't penalize but a human would immediately catch. That's a classic adversarial-example failure mode (Goodhart's law, essentially), and showing it concretely in the phishing-URL domain would be a nice contribution.

A few things that would make this land well:

Predict the artifact types before you see them, then check. Worth hypothesizing upfront what kind of "tells" you'd expect the AI-generated URLs to pick up — things like:

- Overlong or structurally bizarre subdomains that satisfy some lexical feature the model weights heavily but look obviously wrong to a human
- Unnatural keyword stuffing (brand name + urgency word + random token) that reads as "off" linguistically even if it nudges character-level features
- Homoglyphs or encoding tricks that fool string-matching features but render visibly strange
- TLD/path combinations that are statistically rare in the benign class (so the model underweights them) but look nothing like real corporate infrastructure

Then actually check which of these show up. If your intuition is right and the AI finds bypasses that are humanly obvious, that's your headline result. If it instead finds bypasses that are *also* convincing to humans, that's arguably a scarier and equally publishable finding — so the study is interesting either way.

Structure it as three groups, not two, when you run the human eval: real phishing, real benign, and AI-generated-evading. If you only show humans real vs. AI-evading, you can't tell if humans are just pattern-matching "weird = phishing" in general. Having the real phishing baseline shows whether AI-evading URLs get flagged *more* than genuine phishing (interesting — means they're extra suspicious) or *less* (interesting — means the human eval validates the bypass).

Consider a second axis: confidence, not just accuracy. If you can capture how confident/hesitant humans are when they flag something (even a simple 1-5 scale), you might find humans correctly flag the AI-evading URLs but with lower confidence than genuine phishing — suggesting the model found something "off" without hitting a recognizable phishing pattern. That's a nice nuance for the capex argument: training works, but maybe not as cleanly as "human vs model," more like "human catches it but the friction/hesitation cost is different."

This also fixes your novelty concern for the capex conclusion: instead of "training beats filtering" as an assumed premise, your actual argument becomes "the failure mode of automated detection is specifically legible to humans in identifiable ways X, Y, Z" — which is a mechanism-level claim, not just a policy recommendation.

Want to sketch out the human-eval design (sample size, how you'd recruit/administer it, what counts as a flag) or work through what a realistic detector architecture + feature set would look like for the adversarial loop?

Notes

-